

27. Приложения на вградени компютри (embedded systems). Вградени периферни устройства. Методи и модели за проектиране. Примерен код на C.

Вградените компютърни системи (Embedded Systems) са специализирани изчислителни системи, проектирани да изпълняват точно определени, предварително дефинирани задачи в рамките на по-голяма механична или електрическа система. За разлика от компютрите с общо предназначение (PC), те са строго ограничени откъм ресурси (памет, изчислителна мощ, енергопотребление) и често работят в реално време (Real-Time Systems).

1. Приложения на вградените компютри

Вградените системи стоят в основата на съвременната технологична инфраструктура. Техните приложения могат да бъдат класифицирани в няколко основни домейна:

1.1 Автомобилна индустрия

Съвременният автомобил съдържа между 30 и 100 електронни управляващи блока (ECU - Electronic Control Units), свързани в мрежа.

- **Управление на двигател:** Следи сензори за кислород, температура и обороти, за да регулира впръскването на гориво и запалването в реално време.
- **Системи за сигурност:** Пример за такава система в ABS (Antilock Breaking System), ESP, адаптивен круиз и системи за автоматично спиране.

- **Infotainment и комфорт:** Контрол на климатичната инсталация, навигация и мултимедия.

1.2 Индустриална автоматизация

- **Програмируеми логически контролери (PLC):** Надеждни вградени компютри, които управляват поточни линии, роботизирани ръце и производствени процеси.
- **SCADA компоненти:** Сензорни възли (Nodes), които събират данни за налягане, температура у влажност в тежки индустриални условия.

1.3 Смарт домове и Интернет на нещата (IoT - Internet of Things)

- **Домашна автоматизация:** Умни термостати, интелигентни релета (като Shelly), управляеми LED осветления и смарт ключове, интегрирани в системи като Home Assistant.
- **Сензорни мрежи:** Автономни устройства, захранвани от батерии, които комуникират безжично (през WiFi, Zigbee, LoRa) за мониторинг на околната среда.

1.4 Медицинска апаратура

- **Животоподдържащи системи:** Пейсмейкъри (консумацията на енергия е важна - трябва да издържа над 10 години).
- **Диагностична апаратура:** Портативни ЕКГ монитори, помпи за инсулин и пулсоксиметри.

1.5 Потребителска електроника

- Перални машини, роботи-прахосмукачки, рутери и смарт часовници.

2. Вградени периферни устройства

За да комуникира с физическия свят, ядрото на микроконтролера (MCU) разчита на вградени периферни устройства. достъпни чрез регистри за

управление на данни.

2.1 Периферия за общо предназначение

- **GPIO (General Purpose Input Output):** Цифрови пинове, които могат да бъдат конфигурирани софтуерно като входове (четене на бутон или сензор) или изходни (управление на транзистор, LED).
- **Таймери / Броячи:** Хардуерни модули, които броят тактови импулси (вътрешни за измерване на време или външни за броене на събития). Използват се за генериране на прецизни времеви интервали без натоварване на процесора.
- **ШИМ (PWM - Pulse Width Modulation):** Модулация, при която се променя запълването на цифровия сигнал. Прилага се за управление на скоростта на DC мотори, димиране на светлина и генериране на аналогови нива.
- **Watchdog Timer (WDT):** Специализиран таймер за безопасност. Ако софтуерът забие и не нулира този таймер (WDT) в определен интервал, таймерът предизвиква хардуерен рестарт (Reset) на системата.

2.2 Комуникационни интерфейси

Вградените системи използват различни протоколи за обмен на данни:

Протокол	Тип комуникация	Брой проводници	Скорост (типична)	Особености / Приложение
UART	Асинхронна, Point-to-Point	2 (TX, RX)	До 115.2 kbps	Няма тактов сигнал. Използва се за връзка с PC или GPS модули.
I2C	Синхронна, Bus (Master/Slave)	2 (SDA, SCL)	100 kbps - 400 kbps	Адресируема шина. Подходяща за сензори на къси разстояния.
SPI	Синхронна, Bus (Full-Duplex)	4 (MOSI, MISO, SCK, CS)	До няколко Mbps	Много бърза. Използва се за

				SD карти, дисплеи и флаш памети.
CAN	Диференциална, Асинхронна	2 (CAN_H, CAN_L)	До 1 Mbps	Изключително устойчива на смущения. Стандарт в автомобилите.

2.3 Интерфейси за връзка с аналоговия свят

- ADC (Analog-to-Digital Converter): Преобразува аналогово напрежение в цифрова стойност.
- DAC (Digital-to-Analog Converter): Извършва обратното действие - преобразува цифров код в пропорционално аналогово напрежение (напр. за генериране на аудио сигнал).

3. Методи и модели за проектиране

Проектирането на вградени системи изисква едновременно разработване на хардуер (Hardware) и софтуер (Software). Този подход се нарича **Хардуерно-софтуерно съвместно проектиране (HW/SW Co-design)**.

3.1 Модели на жизнения цикъл на разработка

3.1.1 V-модел

V-моделът е разширение на водопадния модел, при което за всеки етап от дефинирането и проектиране има съответна фаза на тестване. Изключително важен при критични за безопасността системи (например в автомобилната индустрия по стандарта ISO 26262).

3.1.2 Agile методологии

Прилагат се при потребителска електроника и IoT, където firmware-ът се обновява често.

3.2 Архитектурни модели на софтуера

3.2.1 Модел с циклично допитване (Polling или Super-Loop)

Най-простата архитектура. Програмата се върти в безкраен цикъл и последователно проверява състоянието на периферните устройства.

- **Предимства:** Лесен за разбиране и имплементация.
- **Недостатъци:** Неефективен разход на енергия, закъснения при реакция, ако някоя функция отнеме твърде много време.

3.2.2 Архитектура, управляване от събития (Event-Driven)

Процесорът се намира в режим на ниска консумация (Sleep). При настъпване на дадено събитие, хардуерът генерира прекъсване. Процесорът спира текущата работа, изпълнява функция за обслужване на прекъсването и отново заспива.

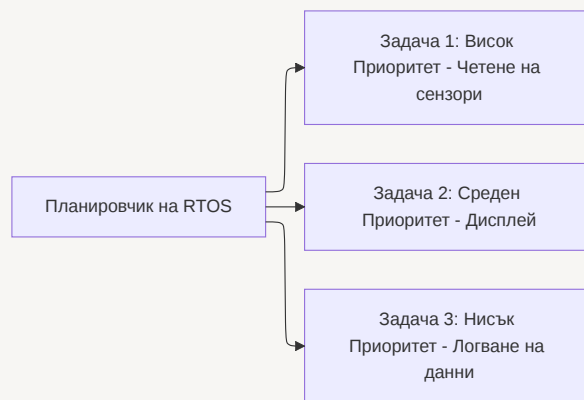
- **Предимства:** Бърза реакция в реално време и ниска консумация на енергия.

3.2.3 Операционни системи за реално време (RTOS)

При сложни системи софтуерът се разделя на отделни нишки/задачи (Tasks) с определен приоритет. Scheduler разпределя процесорното време.

Важно разграничение:

- **Hard Real-Time:** Пропускането на дедлайн води до катастрофален срив на системата (напр. airbag на кола)
- **Soft Real-Time:** Попускането на дедлайн води до разваляне на качеството, но не е критично.



4. Примерен код на C

Следващите примери демонстрират базова работа с периферни устройства на ниско ниво (Bare-metal) за абстрактен 8-битов микроконтролер (подобен на архитектурата AVR/ATmega), където управлението става чрез директен запис в регистрите.

Пример 1: Конфигуриране на GPIO и мигане на светодиод (Polling)

Този пример илюстрира как се задава посока на пин (изход) и как се променя състоянието му в безкраен цикъл.

```
#include <avr/io.h>
#include <util/delay.h>

#define LED_PIN    PB5 // Светодиодът е свързан към пин 5 на П
орт В

int main(void) {
    // 1. Конфигуриране на LED_PIN като ИЗХОД (Output)
    // DDRB е регистърът за посока на Порт В. Записването на
    1 задава изход.
    DDRB |= (1 << LED_PIN);
```

```

while(1) {
    // 2. Включване на светодиода (високо ниво - 5V / 3.3
V)
    PORTB |= (1 << LED_PIN);
    _delay_ms(500); // Софтуерно закъснение от 500 милисе
кунди

    // 3. Изключване на светодиода (ниско ниво - 0V)
    PORTB &= ~(1 << LED_PIN);
    _delay_ms(500);
}

return 0; // Никога не се достига
}

```

Пример 2: Работа с таймер и прекъсване (Timer Interrupt)

Вместо да хабим процесорно време софтуерно с `_delay_ms`, използваме хардуерен таймер, който генерира прекъсване при препълване (Overflow).

```

#include <avr/io.h>
#include <avr/interrupt.h>

#define LED_PIN    PB5

// Функция за обслужване на прекъсването (ISR) при препълване
на Таймер 1
ISR(TIMER1_OVF_vect) {
    // Превключване на състоянието на пина (Toggle) с XOR опе
ратор
    PORTB ^= (1 << LED_PIN);
}

```

```

int main(void) {
    // 1. Конфигуриране на LED_PIN като изход
    DDRB |= (1 << LED_PIN);

    // 2. Настройка на Таймер 1
    // Включваме прекъсване при препълване на таймера
    TIMSK1 |= (1 << TOIE1);

    // Задаваме предделител (Prescaler) 1024. Таймерът започва да брои.
    TCCR1B |= (1 << CS12) | (1 << CS10);

    // 3. Глобално разрешаване на прекъсванията в процесора
    sei();

    while(1) {
        // Процесорът може да изпълнява други задачи тук или да спи
        // Светодиодът ще мига на заден план благодарение на хардуерния таймер
    }

    return 0;
}

```

Пример 3: Проста комуникация по UART (Предаване на данни)

```

#include <avr/io.h>

// Функция за инициализация на UART
void UART_Init(unsigned int ubrr_value) {
    // Задаване на скоростта на трансфер (Baud Rate) чрез регистрите UBRR
}

```

```

UBRR0H = (unsigned char)(ubrr_value >> 8);
UBRR0L = (unsigned char)ubrr_value;

// Разрешаване на предавателя (TX - Transmitter)
UCSR0B = (1 << TXEN0);

// Задаване на рамка на данните: 8 бита данни, 1 стоп бит
UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}

// Функция за изпращане на един байт (символ)
void UART_Transmit(char data) {
    // Изчакване на буфера за предаване да се освободи
    // UDRE0 флагът става 1, когато буферът е празен
    while (!(UCSR0A & (1 << UDRE0)));

    // Поставяне на данните в регистъра, хардуерът ги изпраща
    автоматично
    UDR0 = data;
}

int main(void) {
    // Инициализация на UART със стойност на регистъра за 960
    0 бода (при 16 MHz тактова честота)
    UART_Init(103);

    while(1) {
        UART_Transmit('A'); // Изпращане на символа 'A' непре
        къснато
        _delay_ms(1000);    // На всеки 1 секунда
    }

    return 0;
}

```